

JS Playground User Manual

Table of Contents

JS Playground User Manual.....	1
Table of Contents.....	1
1. Introduction.....	3
1.1 Purpose.....	3
1.2 Intended Audience.....	4
1.3 Scope.....	4
2. System Overview.....	4
3. Account Setup (if applicable).....	5
4. Accessing the System.....	6
5. Navigating the System.....	6
6. Core Features and Functions.....	9
6.1 Sample Code Execution.....	9
6.2 Custom Code Editing and Execution.....	9
6.3 Project Connection.....	9
6.4 Real-time Event Handling.....	9
6.5 Document Modification.....	10
6.6 Preview Output.....	10
6.7 Terminal/Console Output.....	10
7. User Roles and Permissions.....	10
7.1 User Access Levels.....	10
8. Error Messages & Troubleshooting.....	11
8.1 User Not Logged In.....	11
8.2 No Project Connected.....	11
8.3 No Visible Output in Preview.....	12
9. Data & Privacy Information.....	12
10. Contact & Support Information.....	12
11. Appendix.....	13
References.....	13

1. Introduction

The JS Playground is a web-based application designed to support the exploration and use of the Audiotool Nexus Software Development Kit (SDK). The system provides an environment in which users can write, execute, and observe JavaScript code that interacts with Audiotool projects and entities.

The application serves as a lightweight alternative to the full Audiotool Studio interface, enabling users to test SDK functionality, prototype workflows, and understand system behaviour in a controlled and simplified environment.

1.1 Purpose

The purpose of the JS Playground is to provide users with a structured environment for learning and testing the Audiotool Nexus SDK. The system allows users to execute predefined sample scripts or create custom code to interact with Audiotool entities such as devices and project elements.

By offering immediate feedback through code execution and visualization, the application supports rapid prototyping and experimentation. It is particularly useful for understanding SDK workflows before integrating them into full-scale applications.

1.2 Intended Audience

The application is designed for:

- Developers.
- Students learning the SDK.
- Musicians or creators exploring automation tools.

1.3 Scope

The system includes:

- A code editor for writing JavaScript using the Audiotool Nexus SDK.
- A preview environment where users code is executed.
- A collection of sample scripts demonstrating common SDK tasks.
- Tools for authentication and project connection

The system does not:

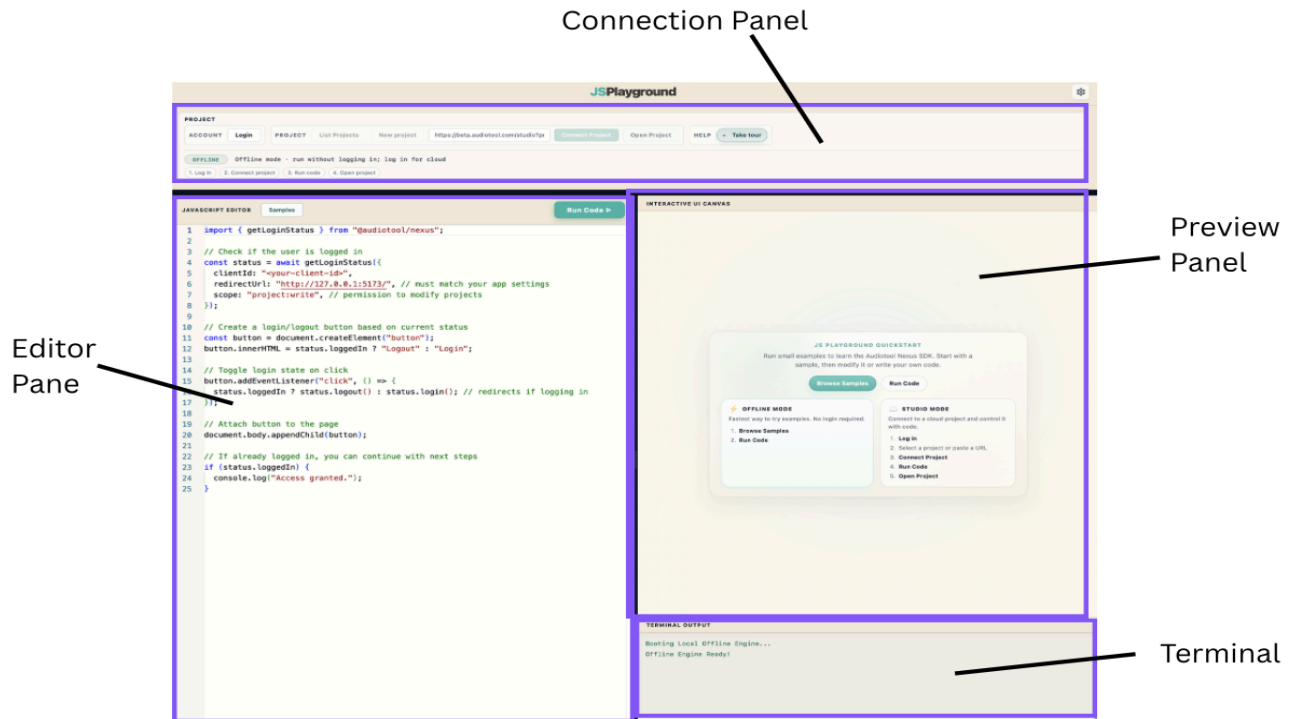
- Provide the full Audiotool Studio (DAW) interface.
- Support full web application development (e.g HTML/CSS editing environments).
- Perform real-time audio rendering within the playground.

2. System Overview

As shown in Figure 1, the JS Playground consists of four primary components:

- Connection Panel
 - Enables users to log in and connect to an Audiotool project.
- Editor Pane
 - Allows users to write and modify JavaScript code.
- Preview Panel
 - Executes the users code in an isolated environment and displays the results.
- Terminal / Console Panel
 - Displays log messages, execution status, and debugging output generated by the code.

Figure 1: JS Playground interface



3. Account Setup (if applicable)

The JS Playground does not manage user accounts directly. Users must create an Audiotool account via:

<https://developer.audiotool.com/>

Once an account is created, users can log in through the Connection Panel to enable project interaction features.

4. Accessing the System

1. Open the JS Playground in a web browser.
2. Navigate to the application URL.

3. Log in using an Audiotool account.
4. Select a sample or enter a custom code.
5. Click “Run Code” to execute the script.

5. Navigating the System

As shown in Figures 1-5, users interact with the system through the following interface elements:

- **Samples Button:** Located within the Editor Pane as shown in Figure 3, the Samples Button loads predefined example scripts into the editor.
- **Editor Pane:** As shown in Figure 3, the Editor Pane allows users to view, modify, and write JavaScript code.
- **Run Code Button:** Also located within the Editor Pane as shown in Figure 3, the Run Code Button executes the current script in the editor.
- **Connection Panel:** As shown in Figure 2, the Connection Panel is used for logging in and connecting to an Audiotool project.
- **Preview Panel:** As shown in Figure 4, the Preview Panel displays visual output generated by the user’s code.
- **Terminal/Console Panel:** As shown in Figure 5, the Terminal/Console Panel displays log messages, execution results, and debugging output (e.g, `console.log()`).

Figure 2: Connection Panel

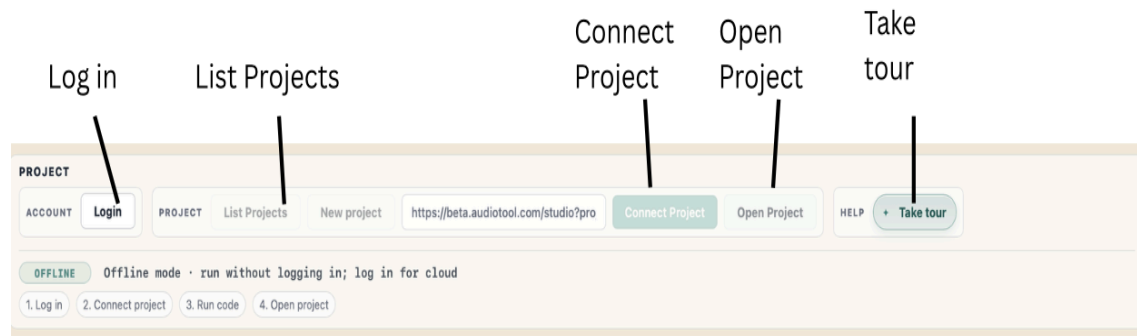


Figure 3: Editor Pane



Figure 4: Preview Panel

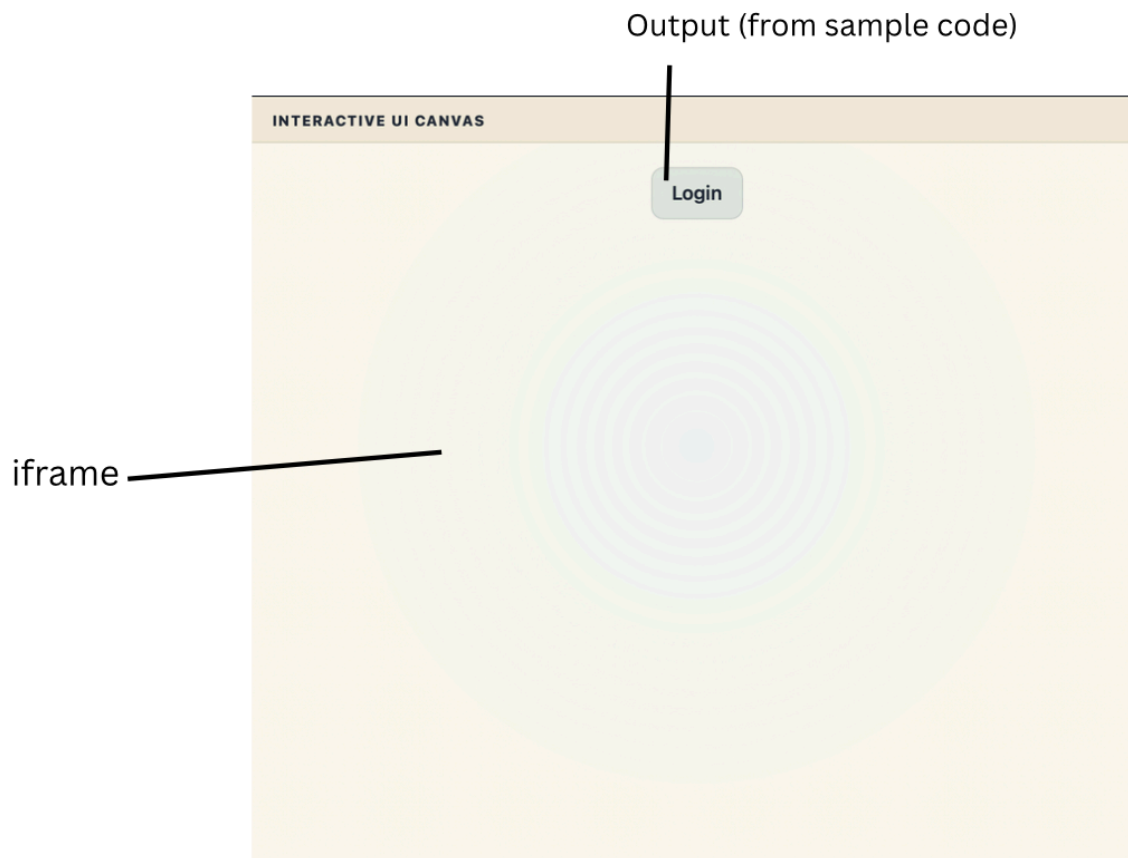
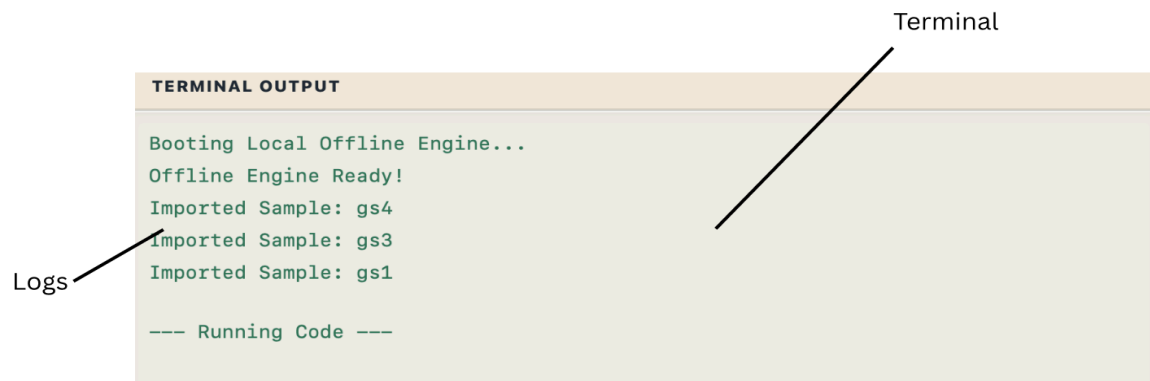


Figure 5: Terminal/ Console Pane



6. Core Features and Functions

The JS Playground provides the following features:

6.1 Sample Code Execution

The system includes a collection of predefined sample scripts that demonstrate common use cases of the Audiotool Nexus SDK. Users can load and run these examples to understand how specific SDK functions work.

Interactive Guide: View the step-by-step walkthrough for sample code execution [here](#).

6.2 Custom Code Editing and Execution

Users can write, modify, and execute their own JavaScript code within the editor. This enables experimentation and testing of SDK functionality in a controlled environment.

Interactive Guide: View the step-by-step walkthrough for custom code editing and execution [here](#).

6.3 Project Connection

Users can log in and connect to an Audiotool project. Once connected, user code can interact with and modify the selected project.

Interactive Guide: View the step-by-step walkthrough for project connection [here](#).

6.4 Real-time Event Handling

The system supports subscribing to SDK events, such as device creation or parameter updates. This allows users to observe how changes in a project are reflected programmatically.

Interactive Guide: View the step-by-step walkthrough for real-time event handling [here](#).

6.5 Document Modification

Users can modify Audiotool project data using SDK functions such as `nexus.modify()`. This includes creating devices, updating properties, and automating project changes.

Interactive Guide: View the step-by-step walkthrough for document modification [here](#).

6.6 Preview Output

The preview panel executes user code in an isolated environment and displays any visual output generated by the script.

Interactive Guide: View the step-by-step walkthrough for preview output [here](#).

6.7 Terminal/Console Output

The system provides a terminal panel that displays log messages, execution results, and debugging information. This is particularly useful for samples that do not produce visual output.

Interactive Guide: View the step-by-step walkthrough for terminal/console output [here](#).

7. User Roles and Permissions

The JS Playground does not include a complex internal role-management system such as administrator, staff, or moderator accounts. However, user capabilities vary depending on whether the user is authenticated and connected to an Audiotool project.

7.1 User Access Levels

User Type	Permissions
Visitor / Unauthenticated User	Browse samples, edit code, run offline-compatible examples, view preview

	output, and view terminal output.
Authenticated User	All visitor permissions, plus log in with an Audiotool account and create an authorized Audiotool client.
Authenticated User with Connected Project	All authenticated-user permissions, plus connect to a project, listen for a project events, and modify project data through SDK.

8. Error Messages & Troubleshooting

This section outlines common issues that users may encounter while using the JS Playground, along with recommended solutions.

8.1 User Not Logged In

Issue: Code that requires authentication (e.g., creating a client or connecting to a project) does not work.

Cause: The user has not logged in using an Audiotool account.

Solution:

1. Use the Connection Panel to log in.
2. Re-run the code after successful authentication.

Interactive Guide: View the step-by-step walkthrough for users not logged in [here](#).

8.2 No Project Connected

Issue: Code that interacts with a project (e.g., creating devices or listening to events) produces no visible effect.

Cause: No Audiotool project has been connected.

Solution:

1. Navigate Connection Panel.
2. Select or enter a valid project URL.
3. Connect to the project before running the code.

Interactive Guide: View the step-by-step walkthrough for no project connected [here](#).

8.3 No Visible Output in Preview

Issue: The preview panel appears empty after running code.

Cause: Some scripts do not generate visual output and instead use console logging.

Solution:

1. Check the Terminal / Console Panel output.
2. Look for messages from `console.log()`.

Interactive Guide: View the step-by-step walkthrough for no visible output in preview [here](#).

9. Data & Privacy Information

The JS Playground does not store or manage user data directly. User authentication and account management are handled externally through Audiotool platform.

10. Contact & Support Information

If you encounter technical issues that are not covered in the Troubleshooting, please reach out to our support team. To help us resolve your issues quickly, please include a brief description of the problem and any error logs from your session.

- Support Email: support@audiotool.com

11. Appendix

- Audiotool Nexus SDK
 - A JavaScript software development kit used to interact with Audiotool projects programmatically.
- Client ID
 - A unique identifier assigned to an application registered on the Audiotool developer platform.
- Project
 - An audiotool workspace containing devices, audio elements, and configurations.
- Device
 - An entity within a project (e.g., Tonematrix, Heisenberg, or effects) that can be created or modified using the SDK.

References

Nexus Audiotool

<https://developer.audiotool.com>